

Abstract Algebra and Number Theory Foundation of cryptography

- ① Prove Fermat's Little Theorem from group-theoretic first principles, then explain how it motivates the Miller-Rabin Probabilistic Primality test. Apply the test (base $a=2$) to $n=561$ and explain why 561 is a Carmichael number that fools the simpler Fermat test.

Answer:

Fermat's little theorem:

If p is a prime and a is not divisible by p , then

$$a^{p-1} \equiv 1 \pmod{p}$$

Proof from group theory:

Consider the multiplicative group

$$G = (\mathbb{Z}/p\mathbb{Z})^*$$

It contains $p-1$ non-zero elements. Since p is prime, every non-zero element has a multiplicative inverse.

For any $a \in G$, the power of a permutes the elements of G . Therefore,

$$a^{p-1} (1 \cdot 2 \cdot 3 \cdots (p-1)) \equiv 1 \cdot 2 \cdot 3 \cdots (p-1) \pmod{p}$$

Since the product is non-zero modulo p , we can cancel it:

$$\boxed{a^{p-1} \equiv 1 \pmod{p}}$$

Thus, Fermat's Little Theorem is proved.

Motivation for Miller-Rabin:

Fermat's theorem suggests a primality test:

$$a^{n-1} \equiv 1 \pmod{n}$$

If this is false, n is definitely composite.

But some composite numbers satisfy this condition for many bases. Therefore, the Fermat test can give false positives.

Miller-Rabin improves this idea by writing

$$n-1 = 2^s d \quad \text{with } d \text{ odd}$$

and checking whether

$$a^d \equiv 1 \pmod{n}$$

or

$$a^{2^r d} \equiv -1 \pmod{n}$$

for some $0 \leq r < s$

If neither condition holds, n is definitely composite.

Apply Miller-Rabin to $n=561$, $a=2$

We have

$$561-1 = 560 = 2^4 \times 35$$

so

$$s=4$$

$$d=35$$

calculate:

$$2^{35} = 269 \pmod{561}$$

Then repeatedly square:

$$2^{70} \equiv 166 \pmod{561}$$

$$2^{140} \equiv 67 \pmod{561}$$

$$2^{280} \equiv 1 \pmod{561}$$

Miller-Rabin requires $2^{35} \equiv 1$, or one of the squaring to become $-1 \equiv 560 \pmod{561}$

Neither happens.

Therefore,

561 is composite

Why 561 fools the Fermat test?

$$561 = 3 \times 11 \times 7$$

So 561 is clearly composite. But 561 is a Carmichael number, meaning

calculate:

$$2^{35} \equiv 269 \pmod{561}$$

Then repeatedly square:

$$2^{70} \equiv 166 \pmod{561}$$

$$2^{140} \equiv 67 \pmod{561}$$

$$2^{280} \equiv 1 \pmod{561}$$

Miller-Rabin requires $2^{35} \equiv 1$, or one of the squaring to become $-1 \equiv 560 \pmod{561}$.
Neither happens.

Therefore,

561 is composite

Why 561 fools the Fermat test?

$$561 = 3 \times 11 \times 7$$

So 561 is clearly composite. But 561 is a Carmichael number, meaning

$$a^{560} \equiv 1 \pmod{561}$$

for every a relatively prime to 561. In particular,

$$2^{560} \equiv 1 \pmod{561}$$

Thus, the simple Fermat test can wrongly say "Probably prime", while Miller-Rabin detects that 561 is composite.

Miller-Rabin is stronger than the Fermat test.

(2) Prove that $\mathbb{Z}_n$, under addition and multiplication modulo n , is a field if and only if n is prime. Illustrate your proof by contrasting the multiplicative structure of $\mathbb{Z}_{11}$ (prime modulus) with $\mathbb{Z}_{12}$ (composite modulus).

Answer:

Theorem:

$\mathbb{Z}_n$ is a field $\iff n$ is prime

Proof:

Case 1: n is prime

If $n = p$ is prime, then every non-zero element of

$$\mathbb{Z}_p = \{0, 1, 2, \dots, p-1\}$$

has a multiplicative inverse modulo p .

This is because for every $a \neq 0$

$$\gcd(a, p) = 1$$

So, by Bezout's identity, there exists b such that

$$ab = 1 \pmod{p}$$

Therefore, every non-zero element has an inverse. Hence,

$$\boxed{\mathbb{Z}_p \text{ is a field}}$$

Case 2: n is composite

Suppose n is composite. Then

$$n = ab, \quad 1 < a, b < n$$

Therefore,

$$a \not\equiv 0, \quad b \not\equiv 0 \pmod{n}$$

But

$$ab = n \equiv 0 \pmod{n}$$

Thus, a and b are non-zero elements whose product is zero. They are called zero divisors.

$$ga(aa) = 1$$

to the present? Actually, there exists b such that

$$ab = 1 \pmod{p}$$

Therefore, every non-zero element has an inverse. Hence,

$$\boxed{\mathbb{Z}_p \text{ is a field}}$$

Case 2: n is composite

Suppose n is composite. Then

$$n = ab, \quad 1 < a, b < n$$

Therefore,

$$a \not\equiv 0, b \not\equiv 0 \pmod{n}$$

but

$$ab \equiv n \equiv 0 \pmod{n}$$

Thus, a and b are non-zero elements whose product is zero. They are called zero divisors.

A field cannot contain zero divisors.

Therefore

$\boxed{\mathbb{Z}_n \text{ is not a field}}$

$\therefore \rightarrow$ Hence

$\mathbb{Z}_n$ is a field iff n is prime

example: $\mathbb{Z}_{11}$ and $\mathbb{Z}_{12}$

$\mathbb{Z}_{11}$ - Prime

11 is prime

non-zero element:
has an inverse
1, 2, ..., 10

✓ Every non-zero element
has an inverse

✓ No zero divisors

example: $2^{-1} = 6$ because

$$2 \times 6 = 12 \equiv 1 \pmod{11}$$

$\mathbb{Z}_{11}$ is a field

$\mathbb{Z}_{12}$ - composite

$12 = 3 \times 4$ is composite

non-zero element:
1, 2, ..., 11

Some element have
no inverse

Has zero divisors

$$3 \times 4 = 12 \equiv 0 \pmod{12}$$

$\mathbb{Z}_{12}$ is not a field

- ③ Construct $\text{GF}(2^3)$ using the irreducible polynomial $x^3 + x + 1$ over $\text{GF}(2)$. Build the multiplication table for a generator element α , and determine whether multiplicative group $\text{GF}(2^3)^*$ is cyclic.

Answer:

Step 1: construct the field

let α be a root of

$$x^3 + x + 1 = 0$$

Therefore,

$$\alpha^3 + \alpha + 1 = 0$$

Since we are working over $\text{GF}(2)$, $-1 = 1$. Hence

$$\alpha^3 = \alpha + 1$$

Every element can be written as

$$a + b\alpha + c\alpha^2, \quad a, b, c \in \text{GF}(2)$$

So the 8 elements are

$$\text{GF}(8) = \{0, 1, \alpha, \alpha^2, \alpha + 1, \alpha^2 + \alpha + 1, \alpha^2 + 1\}$$

Step 2: Power of α (2) (3)

using $\alpha^3 = \alpha + 1$
we get:

$$\alpha^0 = 1$$

$$\alpha^1 = \alpha$$

$$\alpha^2 = \alpha^2$$

$$\alpha^3 = \alpha + 1$$

$$\alpha^4 = \alpha(\alpha + 1) = \alpha^2 + \alpha$$

$$\alpha^5 = \alpha(\alpha^2 + \alpha) = \alpha^3 + \alpha^2 = \alpha^2 + \alpha + 1$$

$$\alpha^6 = \alpha(\alpha^2 + \alpha + 1) = \alpha^3 + \alpha^2 + \alpha = \alpha^2 + 1$$

Finally

$$\alpha^7 = \alpha(\alpha^2 + 1) = \alpha^3 + \alpha = (\alpha + 1) + \alpha = 1$$

$$\therefore \alpha^7 = 1$$

Step 3: Multiplication table of power of α

Since, $\alpha^i \alpha^j = \alpha^{i+j}$

and exponents are taken modulo 7:

x	1	α	α^2	α^3	α^4	α^5	α^6
1	1	α	α^2	α^3	α^4	α^5	α^6
α	α	α^2	α^3	α^4	α^5	α^6	1
α^2	α^2	α^3	α^4	α^5	α^6	1	α
α^3	α^3	α^4	α^5	α^6	1	α	α^2
α^4	α^4	α^5	α^6	1	α	α^2	α^3
α^5	α^5	α^6	1	α	α^2	α^3	α^4
α^6	α^6	1	α	α^2	α^3	α^4	α^5

Step 4: Is $GF(8)^*$ cyclic?

The multiplicative group contains

$$8-1=7$$

non-zero elements

we found

$$\alpha^7 = 1$$

Step 3: Multiplication table of power of α

Since, $\alpha^i \alpha^j = \alpha^{i+j}$

and exponents are taken modulo 7:

x	1	α	α^2	α^3	α^4	α^5	α^6
1	1	α	α^2	α^3	α^4	α^5	α^6
α	α	α^2	α^3	α^4	α^5	α^6	1
α^2	α^2	α^3	α^4	α^5	α^6	1	α
α^3	α^3	α^4	α^5	α^6	1	α	α^2
α^4	α^4	α^5	α^6	1	α	α^2	α^3
α^5	α^5	α^6	1	α	α^2	α^3	α^4
α^6	α^6	1	α	α^2	α^3	α^4	α^5

Step 4: Is $GF(8)^*$ cyclic?

The multiplicative group contains

$$8-1=7$$

non-zero elements

we found

$$\alpha^7 = 1$$

and no smaller positive power of α equals 1. Therefore,

$$\text{ord}(\alpha) = 7$$

Since α generates all 7 non-zero elements,

$$\text{GF}(8)^* = \langle \alpha \rangle$$

Therefore,

$\text{GF}(8)^*$ is cyclic

(4) Derive the time complexity of modular exponentiation via square-and-multiply. Compute $7^{560} \bmod 561$ step by step, and interpret the result in light of 561 being a Carmichael number.

Answer:

Time complexity:

To compute $a^e \pmod{n}$

using square-and-multiply, first write e in binary

If e has k binary bits, we perform at most:

- $k-1$ squaring
- $k-1$ multiplication

Therefore,

$$T(k) = O(k) = O(\log e)$$

so modular exponentiation is much faster than performing e ordinary multiplications.

Compute $7^{560} \pmod{561}$

Since

$$560 = (111000000)_2$$

using repeated squaring:

$$7^2 = 49$$

$$7^4 = 49^2 = 2401 = 157 \pmod{561}$$

$$7^8 = 157^2 = 24649 = 256 \pmod{561}$$

$$7^{16} = 256^2 = 65536 = 460 \pmod{561}$$

$$7^{32} = 460^2 = 211600 = 463 \pmod{561}$$

$$7^{64} = 463^2 = 214369 = 412 \pmod{561}$$

$$7^{128} = 412^2 = 169744 = 421 \pmod{561}$$

$$7^{256} = 421^2 = 177241 = 520 \pmod{561}$$

$$\therefore 560 = 256 + 128 + 64 + 64 + 32 + 16$$

This repeats 64, so the binary decomposition

is better written as

$$560 = 512 + 32$$

Continue:

$$7^{512} = 520^2 = 151 \pmod{561}$$

Therefore,

$$7^{560} = 7^{512} \cdot 7^{32}$$

$$\equiv 151 \cdot 463$$

$$= 69913$$

(128 bits)

$$7^{560} \equiv 1 \pmod{561}$$

$$7^{128} = 412^2 = 169744 = 421 \pmod{561}$$

$$7^{256} = 421^2 = 177241 = 520 \pmod{561}$$

$$\therefore 560 = 256 + 128 + 64 + 64 + 32 + 16$$

This repeats 64, so the binary decomposition

is better written as

$$560 = 512 + 32$$

Continue:

$$7^{512} = 520^2 = 151 \pmod{561}$$

Therefore,

$$7^{560} = 7^{512} \cdot 7^{32}$$

$$= 151 \times 463$$

$$= 69913$$

$$7^{560} \equiv 1 \pmod{561}$$

Interpretation:

Since

$$561 = 3 \times 11 \times 17$$

561 is composite but

$$7^{560} \equiv 1 \pmod{561}$$

This is exactly the type of behavior that makes Carmichael numbers dangerous for the simple Fermat primality test.

Thus, the Fermat test may incorrectly say that 561 is probably prime, even though it is composite.

Carmichael number 561 can fool the Fermat test.

⑤ Compare deterministic (AKS) and probabilistic (Miller-Rabin) primality testing in terms of asymptotic complexity, error probability, and practical relevance to RSA key generation at 2048-bit and 4096-bit key sizes.

Answer:

Feature	AKS	Miller-Rabin
Type	Deterministic	Probabilistic
Complexity	Polynomial time, roughly $O((\log n)^6)$	Very fast, about $O(k(\log n)^3)$ for k rounds
Error	No error	Very small probability of false prime
Practical use	Mainly theoretical	Widely used in practice, including RSA
RSA 2048/4096-bit	Too slow compared with practical test	Highly suitable and efficient

⑤ Compare deterministic (AKS) and probabilistic (Miller-Rabin) primality testing in terms of asymptotic complexity, error probability, and practical relevance to RSA key generation at 2048-bit and 4096-bit key sizes.

Answer:

Feature	AKS	Miller-Rabin
Type	Deterministic	Probabilistic
Complexity	Polynomial time, roughly $O((\log n)^6)$	Very fast, about $O(k(\log n)^3)$ for k rounds
Error	No error	Very small probability of false prime
Practical use	Mainly theoretical	Widely used in practice, including RSA
RSA 2048/4096-bit	Too slow compared with practical test	Highly suitable and efficient

Error Probability

For a suitable Miller-Rabin, the probability of incorrectly accepting a composite number is at most $\frac{1}{4}$.

After k independent rounds:

$$P(\text{error}) \leq \left(\frac{1}{4}\right)^k$$

So, with enough rounds, the error probability becomes extremely small.

For 2048-bit and 4096-bit RSA, Miller-Rabin is preferred because it is much faster and its error probability can be made extremely small by using several rounds.

Modular Arithmetic & the Chinese Remainder Theorem

- ⑥ Derive the Extended Euclidean Algorithm formally (as a sequence of linear combinations) then use it to compute $\gcd(240, 46)$ together with the Bezout's co-efficients x, y such that $240x + 46y = \gcd(240, 46)$

Answer:

Formal Derivation:

The Euclidean Algorithm uses repeated division:

$$a = q_1 b + r_1$$

$$b = q_2 r_1 + r_2$$

$$r_1 = q_3 r_2 + r_3$$

Continue until the remainder becomes 0. The last zero remainder is the gcd.

The Extended Euclidean Algorithm expresses every remainder as a linear combination of a and b .

$$r_i = x_i \cdot a + y_i \cdot b$$

starting with

$$r_0 = a, r_1 = b$$

we have

$$x_0 = 1, y_0 = 0$$

$$x_1 = 0, y_1 = 1$$

and recursively,

$$x_i = x_{i-2} - q_i x_{i-1}$$

$$y_i = y_{i-2} - q_i y_{i-1}$$

Thus, for the final non-zero remainder,

$$ax + by = \gcd(a, b)$$

Compute $\gcd(240, 46)$:

using Euclidean Algorithm:

$$240 = 46(5) + 10$$

$$46 = 10(4) + 6$$

$$10 = 6(1) + 4$$

$$6 = 4(1) + 2$$

$$4 = 2(2) + 0$$

Therefore, $\gcd(240, 46) = 2$

Find Bezout co-efficients:

Back-substitute:

$$2 = 6 - 4$$

$$\text{Since } 4 = 10 - 6$$

$$\text{we get, } 2 = 6 - (10 - 6) = 2(6) - 10$$

Since

$$6 = 46 - 4(10)$$

$$2 = 2(46 - 4(10)) - 10$$

$$= 2(46) - 9(10)$$

Compute $\gcd(240, 46)$:

using Euclidean Algorithm:

$$240 = 46(5) + 10$$

$$46 = 10(4) + 6$$

$$10 = 6(1) + 4$$

$$6 = 4(1) + 2$$

$$4 = 2(2) + 0$$

Therefore, $\gcd(240, 46) = 2$

Find Bezout co-efficients:

Back-substitute:

$$\text{Since, } 2 = 6 - 4$$

$$4 = 10 - 6$$

we get,

$$2 = 6 - (10 - 6) = 2(6) - 10$$

Since

$$6 = 46 - 4(10)$$

$$2 = 2(46 - 4(10)) - 10$$

$$= 2(46) - 9(10)$$

Since

$$10 = 240 - 5(46)$$

$$2 = 2(46) - 9(240 - 5(46))$$

$$2 = -9(240) + 47(46)$$

Therefore,

$$x = -9, y = 47$$

$$\therefore \gcd(240, 46) = 2, x = -9, y = 47$$

⑦ state and Prove the Chinese Remainders Theorem (existence and uniqueness) for pairwise coprime moduli. Apply it to solve: $x \equiv 2 \pmod{3}$, $x \equiv 3 \pmod{5}$, $x \equiv 4 \pmod{7}$, $x \equiv 5 \pmod{11}$

Answer:

statement:

If $m_1, m_2, \dots, m_k$ are pairwise coprime, then the system

$$x \equiv a_i \pmod{m_i}$$

has a unique solution modulo

$$M = m_1 m_2 \dots m_k$$

Proof:

Let

$$M = m_1 m_2 \dots m_k \Rightarrow M_i = \frac{M}{m_i}$$

Since the moduli are pairwise coprime
 $\gcd(M_i, m_i) = 1$

Therefore, M_i has an inverse y_i modulo m_i :

$$M_i y_i \equiv 1 \pmod{m_i}$$

$$\text{Take } x = \sum_{i=1}^k a_i M_i y_i$$

$$\text{Then } x \equiv a_i \pmod{m_i}$$

for every i . Hence, a solution exists.

If x and x' are two solutions, then

$$x - x' \equiv 0 \pmod{m_i}$$

for every i . Since the moduli are pairwise coprime

$$M \mid (x - x')$$

Thus,
$$\boxed{x \equiv x' \pmod{M}}$$

So the solution is unique modulo M .

Application

Given
$$\begin{aligned} x &\equiv 2 \pmod{3}, & x &\equiv 3 \pmod{5}, & x &\equiv 6 \pmod{7} \\ x &\equiv 5 \pmod{11} \end{aligned}$$

Here, $M = 3 \times 5 \times 7 \times 11 = 1155$

We can solve step by step:

From the first two: $x \equiv 2 + 3k$

Using $x \equiv 3 \pmod{5}$:

$$\begin{aligned} 2 + 3k &\equiv 3 \pmod{5} \\ 3k &\equiv 1 \pmod{5} \end{aligned}$$

Since $3^{-1} \equiv 2 \pmod{5}$, $k \equiv 2 \pmod{5}$

So, $x \equiv 8 \pmod{15}$

Now use $x \equiv 4 \pmod{7}$;

$$8 + 15k \equiv 4 \pmod{7}$$

$$1 + k \equiv 4 \pmod{7}$$

$$k \equiv 3 \pmod{7}$$

Thus,

$$x \equiv 53 \pmod{105}$$

Finally use $x \equiv 5 \pmod{11}$

$$53 + 105k \equiv 5 \pmod{11}$$

$$9 + 6k \equiv 5 \pmod{11}$$

$$6k \equiv 7 \pmod{11}$$

$$\text{Since } 6^{-1} \equiv 2 \pmod{11}$$

$$k \equiv 14 \equiv 3 \pmod{11}$$

Therefore,

$$x = 53 + 105(3) = 368$$

$$x \equiv 368 \pmod{1155}$$

So the smallest positive solution is: $x = 368$.

Now use $x \equiv 4 \pmod{7}$;

$$8 + 15k \equiv 4 \pmod{7}$$

$$1 + 1k \equiv 4 \pmod{7}$$

$$k \equiv 3 \pmod{7}$$

Thus,

$$x \equiv 53 \pmod{105}$$

Finally

use $x \equiv 5 \pmod{11}$

$$53 + 105k \equiv 5 \pmod{11}$$

$$9 + 6k \equiv 5 \pmod{11}$$

$$6k \equiv -4 \pmod{11}$$

Since $6^{-1} \equiv 2 \pmod{11}$

$$k \equiv 14 \equiv 3 \pmod{11}$$

Therefore,

$$x = 53 + 105(3) = 368$$

$$x \equiv 368 \pmod{1155}$$

So the smallest positive solution is: $x = 368$.

⑧ Derive the CRT-based RSA decryption/signing optimization from first principles (computing d_p , d_q and q_{inv}), and demonstrate the $\sim 4\times$ speed-up with a fully worked numerical example using small primes p, q .

Answer:

For RSA, $n = pq$, $d = e^{-1} \pmod{\phi(n)}$

Instead of calculating $m = c^d \pmod n$, we

$$d_p = d \pmod{p-1}$$

$$d_q = d \pmod{q-1}$$

$$q_{inv} = q^{-1} \pmod p$$

Then, $m_1 = c^{d_p} \pmod p$

$$m_2 = c^{d_q} \pmod q$$

$$h = q_{inv} (m_1 - m_2) \pmod p$$

$$\boxed{m = m_2 + qh}$$

This gives the same result as $c^d \pmod n$.

Example

Take $p=5, q=11$

$$n=55, \phi(n)=40$$

Choose $e=3$ so $d=27$

Therefore,

$$dp = 27 \bmod 4 = 3$$

$$dq = 27 \bmod 10 = 7$$

$$d_{inv} = 11^{-1} \bmod 5 = 1$$

For $c = m$

$$m_1 = 12^3 \bmod 5 = 3$$

$$m_2 = 12^7 \bmod 11 = 8$$

$$h = (m_2 - 8) \bmod 5 = 2$$

$$\therefore m = 8 + 11(2) = 30.$$

Speed up:

CRT uses two smaller exponentiations instead of one large one, giving approximately

CRT $\approx 4 \times$ Speed $4 \times$ speed-up.

Example

Take $p=5$, $q=11$

$$n=55, \quad \phi(n)=40$$

Choose $e=3$, so $d=27$

Therefore,

$$dp = 27 \bmod 4 = 3$$

$$dq = 27 \bmod 10 = 7$$

$$a_{inv} = 11^{-1} \bmod 5 = 1$$

For $c = 12$

$$m_1 = 12^3 \bmod 5 = 3$$

$$m_2 = 12^7 \bmod 11 = 8$$

$$h = (m_1 - 8) \bmod 5 = 2$$

$$\therefore m = 8 + 11(2) = 30$$

Speed up:

CRT uses two smaller exponentiations instead of one large one, giving approximately

CRT is 4x speed

4x speed-up.

⑨ Explain the Bellcore fault-injection attack on RSA-CRT signatures: how a single fault modular computation during signing leaks a factor of n , and what countermeasures (e.g. signature verification before release) prevent it.

Answer:

In RSA-CRT signing, the signer computes:

$$S_p = m^d \pmod{p}$$

$$S_q = m^d \pmod{q}$$

and combines them using CRT to produce the signature s .

Attack:

An attack induces a fault in one computation, for example S_p , producing a fault signature s' .

The correct signature satisfies:

$$s^e \equiv m \pmod{n}$$

but the faulty signature generally satisfies this only modulo one prime.

Therefore, the attacker can calculate:

$$\boxed{\gcd(s^e - m, n)}$$

If the fault affected the p-side, this gcd typically gives: $\boxed{p}$

Then the attacker obtains the other factor: $q = n/p$.

Thus, the private RSA key can be compromised.

Counter-measures

- verify the signature before releasing it:

$$s^e \equiv m \pmod{n}$$

- use fault detection/redundant computation.

- use blinding and protected CRT implementations.

⑩ Compute a modular inverse using the Extended Euclidean Algorithm and Fermat's Little Theorem based exponentiation (for prime modulus), and discuss the trade-offs between the two approaches for large modulus.

Answer:

Find the inverse of modulo 11.

we need x such that

$$3x \equiv 1 \pmod{11}$$

Method 1: Extended Euclidean Algorithm

$$11 = 3(3) + 2$$

$$3 = 2(1) + 1$$

Back-substitute:

$$1 = 3 - 2$$

$$= 3 - (11 - 3 \times 3)$$

$$= 4(3) - 11$$

Thus, $3(4) \equiv 1 \pmod{11}$

$$\therefore 3^{-1} \equiv 4 \pmod{11}$$

Method 2: Fermat's Little Theorem

Since 11 is prime

$$a^{p-1} \equiv 1 \pmod{p}$$

Therefore, $a^{-1} = a^{p-2} \pmod{p}$

So, $3^{-1} \equiv 3^9 \pmod{11}$

using repeated squaring:

$$3^2 = 9$$

$$3^4 = 4$$

$$3^8 = 5$$

$$\therefore 3^9 = 5 \times 3 = 15 \equiv 4 \pmod{11}$$

$$\therefore 3^{-1} \equiv 4 \pmod{11} //$$

Comparison

EEA

Fermat

Work for any a, n when
 $\gcd(a, n) = 1$

Requires prime modulus

usually more efficient
for one inverse

Requires modular
exponentiation

Gives Bezout coefficient
for

Simple formula a^{p-2}

Comparison

EEA

Fermat

Work for any a, n when
 $\gcd(a, n) = 1$

Requires prime modulus

usually more efficient
for one inverse

Requires modular
exponentiation

Gives Bezout coefficient

Simple formula a^{p-2}



Classical & Symmetric Block Ciphers

- Q11 Design and fully execute a playfair cipher— encryption and decryption of a phrase of your choosing (minimum 16 letters) with a keyword of your choosing. Present the complete 5x5 key matrix as a table, and show every digraph substitution step.

Answer:

Playfair Cipher

Key word: MONARCHY

plain-text: MEET ME AT THE OLD BRIDGE

After removing space: MEETMEATTHEOLDBRIDGE

Step 1: Construct the 5x5 Key Matrix

combine the keyword with the remaining alphabet, omitting J.

M O N A R

C H Y E T

L D B I G

F K P Q S

V W X Z

Step 2: Divide Plaintext into Digraphs

ME ET ME AT TH EO LD BR ID GE

No repeated-letter pairs occurs, so no filler X is needed.

Step 3: Encryption Rules

- Same row: take the letter to the right
- Same column: take the letter below
- Different row and column: form a rectangle and take the letters in the same row but opposite columns.

M O N A P

C H Y E T

L D B I G

F K P Q S

V W X Z

Step 2: Divide Plaintext into Digraphs

ME ET ME AT TH EO LD BR ID GE

No repeated-letter pairs occurs, so no filler
X is needed.

Step 3: Encryption Rules

- Same row: take the letter to the right
- Same column: take the letter below
- Different row and column: form a rectangle and take the letters in the same row but opposite columns.

Step 4: Encryption

plain pair	Cipher pair	Rule
ME	AC	Rectangle
ET	TC	Rectangle
ME	AC	"
AT	RE	"
TH	CY	"
EO	AA	"
LD	DB	same column
BR	GN	"
ID	GB	"
GE	IT	"

Therefore,

Ciphertext = ACTCACRECYHAADBGNGBIT

Step 5: Decryption

For decryption, reverse the rules: move left for the same row and up for the same column.

Cipher pairs	plain pairs
AC	ME
TC	ET
AE	ME
RE	AT
CF	TH
HA	EO
DB	LD
GN	OR
GB	ID
IT	GE

Thus,

MEET ME AT THE OLD BRIDGE

So the original message recovers:

MEET ME AT THE OLD BRIDGE

- (12) Critically analyze the structural weakness of the playfair cipher against the digraph frequency analysis and explain why it remains vulnerable to known-plaintext attacks despite being far stronger than monoalphabetic substitution.

Answer:

Playfair cipher - Weakness

- Playfair encrypts two letters at a time, so it is stronger than monoalphabetic substitution. However, it has some weaknesses:
- Common digraphs can still create frequency patterns
 - The same plaintext digraph always gives the same ciphertext digraph.

• Repeated patterns may reveal information about the plaintext.

• In a known-plaintext attack, known plaintext-ciphertext can help an attacker find the key square.

(13) Derive and trace two full rounds of a simplified Feistel-structured cipher (DES-like), showing $L_0, R_0 \rightarrow L_1, R_1 \rightarrow L_2, R_2$ symbolically, then with the numeric values $L_0 = 0xAAAAAAAA, R_0 = 0xF0F0F0F0, K_1 = 0x33333333$, using XOR as the round function.

Answer:

Symbolic derivation:

$$L_i \Rightarrow R_{i-1}$$

$$R_i = L_{i-1} \text{ XOR } F(R_{i-1}, K_i)$$

Here,

$$F(R, K) = R \text{ XOR } K$$

Round 1

$$L_0 = \text{AAAAAAAA}$$

$$R_0 = \text{FOFOFOFO}$$

$$K_1 = \text{OFOFOFOF}$$

$$F(R_0, K_1):$$

$$\text{FOFOFOFO} \text{ XOR } \text{OFOFOFOF} = \text{FFFFFFFF}$$

Therefore,

~~$$L_1 = L_0 \text{ XOR } K_1$$~~

$$L_1 = R_0 = \text{FOFOFOFO}$$

$$R_1 = L_0 \text{ XOR } F(R_0, K_1)$$

$$\begin{aligned} &= \text{AAAAAAAA} \text{ XOR } \text{FFFFFFFF} \\ &= \text{55555555} \end{aligned}$$

Here,

$$F(R, K) = R \text{ XOR } K$$

Round 1

$$L_0 = \text{AAAAAAAAAA}$$

$$R_0 = \text{FOFOFOFO}$$

$$K_1 = \text{OFOFOFOF}$$

$$F(R_0, K_1):$$

$$\text{FOFOFOFO} \text{ XOR } \text{OFOFOFOF} = \text{FFFFFFFF}$$

Therefore,

~~$$L_1 = L_0 \text{ XOR } P(R_0, K_1)$$~~

$$L_1 = R_0 = \text{FOFOFOFO}$$

$$R_1 = L_0 \text{ XOR } P(R_0, K_1)$$

$$\begin{aligned} &= \text{AAAAAAAAAA} \text{ XOR } \text{FFFFFFFF} \\ &= \text{55555555} \end{aligned}$$

Round 2

$$K_2 = 33333333$$

$F(R_1, K_2):$

$$55555555 \text{ XOR } 33333333 = 66666666$$

Therefore,

$$L_2 = R_1 = 55555555$$

$$R_2 = L_1 \text{ XOR } F(R_1, K_2)$$

$$= \text{FOFOFOFO} \text{ XOR } 66666666$$

$$= 96969696$$

Final answer

$$L_0, R_0 \rightarrow L_1, R_1 \rightarrow L_2, R_2$$

$$\text{AAAAAAAA, FOFOFOFO} \rightarrow \text{FOFOFOFO}$$

$$55555555 \rightarrow 55555555, 96969696$$

Round 2

$$K_2 = 33333333$$

$F(R_1, K_2):$

$$55555555 \text{ XOR } 33333333 = 66666666$$

Therefore,

$$L_2 = R_1 = 55555555$$

$$R_2 = L_1 \text{ XOR } F(R_1, K_2)$$

$$= \text{FOFOFOFO} \text{ XOR } 66666666 \\ = 96969696$$

Final answer

$$L_0, R_0 \rightarrow L_1, R_1 \rightarrow L_2, R_2$$

$$\text{AAAA AAAA}, \text{FOFOFOFO} \rightarrow \text{FOFOFOFO}$$

$$55555555 \rightarrow 55555555, 96969696$$

(14) Formally define the avalanche effect in block cipher. Using a toy 8-bit substitution-permutation network of your own construction, demonstrate how flipping a single input bit changes at least half the output bits, and explain why this property is essential for cryptographic security.

Answer:

The avalanche effect means that a small change in the input, such as flipping one bit, should cause a large change in the ciphertext.

Ideally, changing one input bit changes about 50% of the output bits.

Toy example

suppose an 8-bit cipher gives:

Input: 10110010

Output: 01101101

Now flip only the first input bit:

00110010

suppose the new output is:

11010011

compare:

old: 01101101

New: 11010011

↑↑ ↑ ↑↑

Here 5 out of 8 bits changed.

$$\frac{5}{8} = 62.5\%$$

so the toy cipher demonstrates the avalanche effect.

Important %

Avalanche effect makes it difficult for attackers to find relationships between plaintext and ciphertext. It helps provide confusion and diffusion, which are essential for secure block ciphers.

- (15) Evaluate the brute-force resistance of an 8-character alphanumeric password (62-character alphabet) versus a 6-digit PIN, under a modern GPU cracking rate of 10^9 guesses/second. Compute worst-case and expected-case cracking times for both, and discuss the implications for password policy design.

Answer %

Given:

Cracking rate = 10^9 guesses/second

A) 8-character alphanumeric password

Alphabet size = 62

Total possibilities:

$$62^8 = 218,340,105,584,896$$

Worst case time:

$$218,340,105,584,896 / 10^9$$

$$= 218,340 \text{ seconds}$$

$$= 2.53 \text{ days}$$

Expected - case time $\approx$ half

$$1.26 \text{ days}$$

B) 6-digit PIN

Each position has 10 possibilities.

Total possibilities:

$$10^6 = 1,000,000$$

Worst-case:

Given:

Cracking rate = 10^9 guesses/second

Ⓐ 8-character alphanumeric password

Alphabet size = 62

Total possibilities:

$$62^8 = 218,340,105,584,896$$

Worst case time:

$$218,340,105,584,896 / 10^9$$

$$= 218,340 \text{ seconds}$$

$$= 2.53 \text{ days}$$

Expected - case time $\approx$ half

$$1.26 \text{ days}$$

Ⓑ 6-digit PIN

Each position has 10 possibilities.

Total possibilities:

$$10^6 = 1,000,000$$

Worst-case:

$$T_{\text{worst}} = \frac{10^6}{10^9} = 0.001 \text{ sec} \\ = 1 \text{ ms}.$$

Expected - case

$$T_{\text{avg}} = \frac{10^6}{2 \times 10^9} = 0.0005 \text{ sec} \\ = 0.5 \text{ ms}.$$

Comparison

Credential	Possibilities	Worst case	Expected case
8-char alphanumeric	$62^8 \approx 2.18 \times 10^{14}$	$\approx 2.53 \text{ days}$	1.26 days
6-digit pin	10^6	1ms	0.5 ms

Implications

Longer passwords with larger character sets provide much greater brute force resistance. Password policies should encourage long, unique password, while PINs should have additional protections such as rate limiting and account lockout.

Hash Functions & Lattice-Based (Post-Quantum) Cryptography

31) Formally define preimage resistance, second-preimage resistance, and collision resistance for a cryptographic hash function. Derive the birthday-attack complexity ($\approx 2^{n/2}$) for finding a collision in an collision in an n -bit hash, and explain why this motivates 256-bit (rather than 128-bit) output lengths for modern hash functions.

Answer:

Let hash function be:

$$H: \{0, 1\}^* \rightarrow \{0, 1\}^n$$

where output size = n bits

Preimage resistance:

Given a hash value, it should be difficult

to find any message m such that:
 $H(m) = y$

Second-Preimage Resistance

Given a message m_1 , it should be difficult to find another message $m_2 \neq m_1$ such that:
 $H(m_1) = H(m_2)$

Collision Resistance

It should be difficult to find any two different messages:

$$m_1 \neq m_2$$

such that: $H(m_1) = H(m_2)$

Birthday Attack

For an n -bit hash, there are:

2^n possible hash values

Due to the birthday paradox, a collision can be found after approximately

$2^{n/2}$ attempts.

Therefore:

• 128-bit hash $\rightarrow$ about 2^{64} collision attempts

• 256-bit hash $\rightarrow$ about 2^{128} collision attempts

So modern hashes use 256-bit outputs to provide approximately 128-bit collision security.

$2^{n/2}$ attempts.

Therefore:

• 128-bit hash $\rightarrow$ about 2^{64} collision attempts

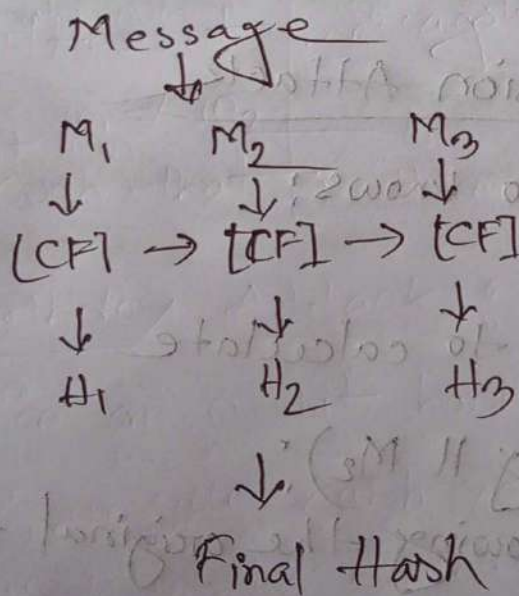
• 256-bit hash $\rightarrow$ about 2^{128} collision attempts

So modern hashes use 256-bit outputs to provide approximately 128-bit collision security.

32) Trace the Merkle-Damgård construction with a diagram showing message blocks, compression function calls, and chaining variables. Explain the length attack that this construction is vulnerable to, and how HMAC and SHA-3 (sponge construction) avoid it.

Answer:

Basic diagram



Model formally:

$$IV + M_1 \xrightarrow{CF} H_1$$

$$H_1 + M_2 \xrightarrow{CF} H_2$$

$$H_2 + M_3 \xrightarrow{CF} H_3$$

↓
hash value

CF = Compression function

Each message block updates the previous chaining value.

Length - Extension Attack

An attacker who knows:

$H(M)$:

may be able to calculate

$H(M \parallel \text{padding} \parallel M_2)$:

without knowing the original message M .

This happens because the hash value acts

as the internal chaining state.

How to avoid it?

HMAC

Secret key + Message



HMAC



Authentication

HMAC uses a secret key and does not expose the internal hash state

SHA-3: uses a sponge construction, not Merkle-Damgård, so the traditional length extension attack does not apply.

(25) Explain the core idea behind CRYSTALS - Kyber

Module - learning - with - Errors (Module - LWE)

Key encapsulation mechanism, including how "noise" is used to hide the secret, and why this construction resists both classical and quantum attack.

Answer: